Aether UAV:

Search, Sense, & Surveying Drone

By: Daniel Rushakoff, Nasser Trad, Chris Bell

4830 Advisor: James Kang

December 15, 2025

What is Aether UAV:

- Remote-controlled multifunctional quadcopter architecture designed for safe, remote environmental monitoring in disaster zones.
- Core Mission - Replace human presence with robotic autonomy in hazardous environments (e.g., chemical spills, industrial accidents).

Project Overview:

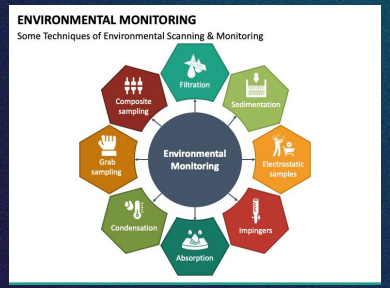
Core Functions:

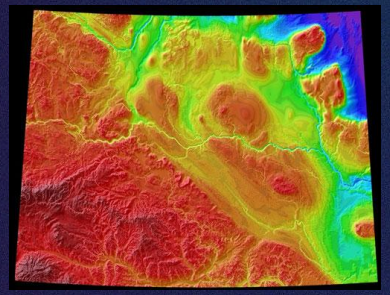
- Search: Locate humans/pets or anything.
- Sense: Detect radiation, hazardous gases, air quality, and collect weather data.
- Conduct remote surveillance in rough terrain, eliminating the need for physical human presence.

Use Cases & Motivation:

- Search and Rescue in natural disasters
- Environmental monitoring (chemical spills, nuclear reactor accidents, fires, earthquakes, tsunamis)
- Remote terrain mapping for research and development (volcanoes, caves, sinkholes, snowy/rugged mountain ranges, deserts, arctic, general uninhabited areas/regions)
- Real-time data collection for disaster response & assessment, for future safety precautions.







Hardware Overview:

Flight:

- Quadcopter Frame - F450 Frame w/ Landing Skid
- Brushless Motors (x4) - Standard 2212 920kV (CW/CCW)
- Electronic Speed Controllers (x4) - 20A ESC’s (included)
- Propellers - 10x4.5 , 9450 Propellers
- Battery (Li-Po) - Dual-Battery Architecture :

  Flight Domain: 11.1V 4200mAh

  Compute Domain: 11.1V 2200mAh
- 5V/5A Buck Converter

Navigation & Positioning:

- M8N GPS Module with Mounting Bracket
- Internal IMU and Compass

















Hardware Overview:





Onboard Processing & Control:

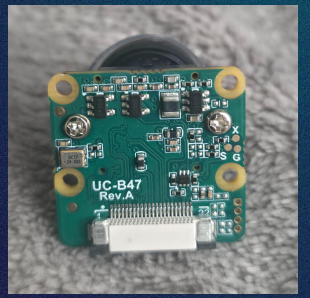
- Flight Controller - Pixhawk 2.4.8 w/ Vibration Dampening Plate
- Onboard Computer - Raspberry Pi 5
- Storage Module - SanDisk SD Cards



Search Sensors:

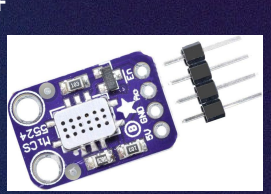
- HD Visual Camera (front camera) - Arducam for Raspberry PI Camera 12 MP Module 3 Wide, 120˚ IMX708

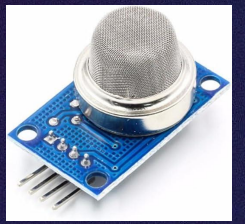


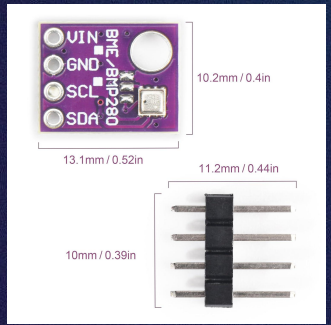


Sense Sensors (Environmental):

- Gas Sensors (CO/NOx) - MiCS-5524
- Air Quality Sensor (Smoke/LPG) - MQ-2
- Temperature Sensor - BME280
- Pressure Sensor - BME280
- Humidity Sensor - BME280



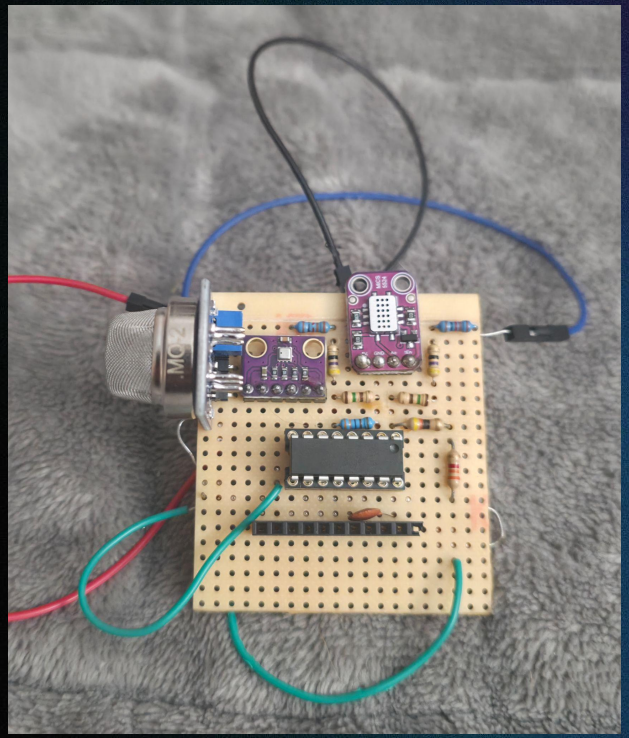


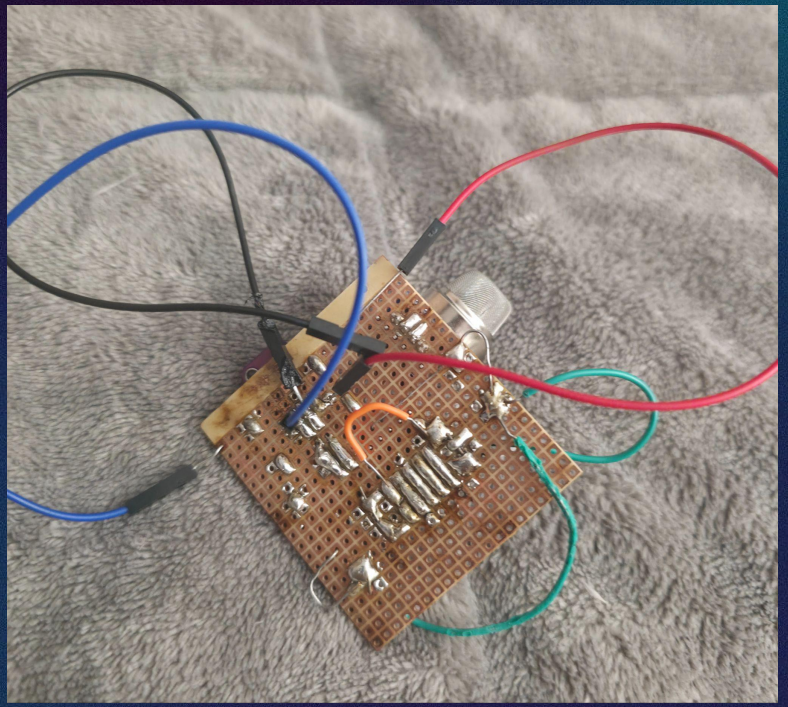


Hardware Overview:

Sense Sensors (Environmental):

- ADC - MCP3008 and 1.8kΩ,3.3kΩ resistors





Hardware Overview:

Survey Sensors (Mapping):

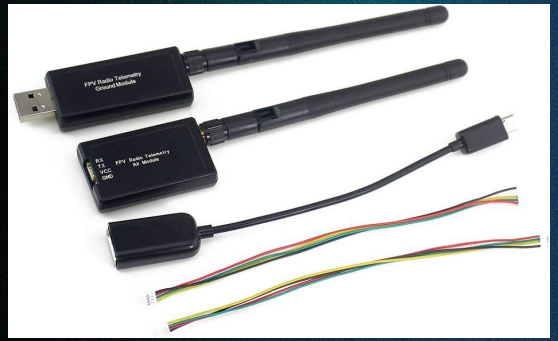
- Visual Surveying: Bottom-facing Arducam for terrain i
- Arducam 8MP Sony IMX219





Communication System:

- Telemetry Radio - 915MHz 3DR 500MW
- RC System - FS-i6X transmitter (10 CH),
  FS-iA6B receiver





Search Module:



System:

- Forward-Facing Visual Search.

Hardware:

- Arducam Wide (120˚ FOV) mounted on the nose.

Function:

- This camera mimics a pilot's eyes. It scans ahead to find specific objects (survivors, vehicles, fire smoke) in the distance.
- Scans the horizon to locate targets (survivors, hazards) and navigate obstacles in real-time.



Example:

- Used in rescue operations to detect survivors under debris or in dark areas

Survey Module:



System:

- Downward-Facing Terrain Survey.

Hardware:

- Arducam 8MP V2.3 mounted on the undercarriage.

Function:

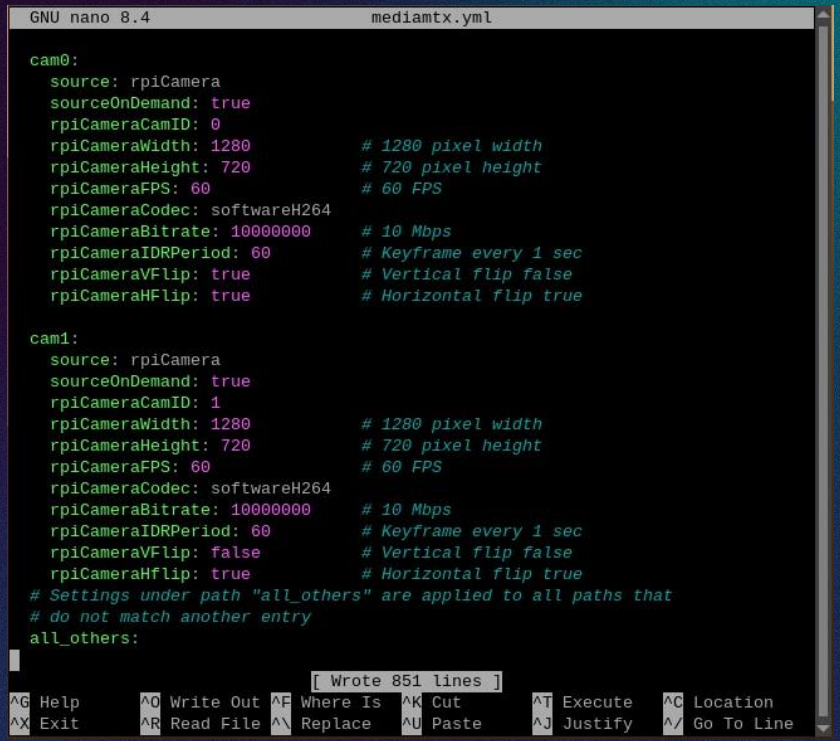
- Captures high-angle aerial imagery to view terrain conditions, assess flood/debris spread, and document safe landing zones.
- Real-time or post-mission data export



Live Video Feeds:

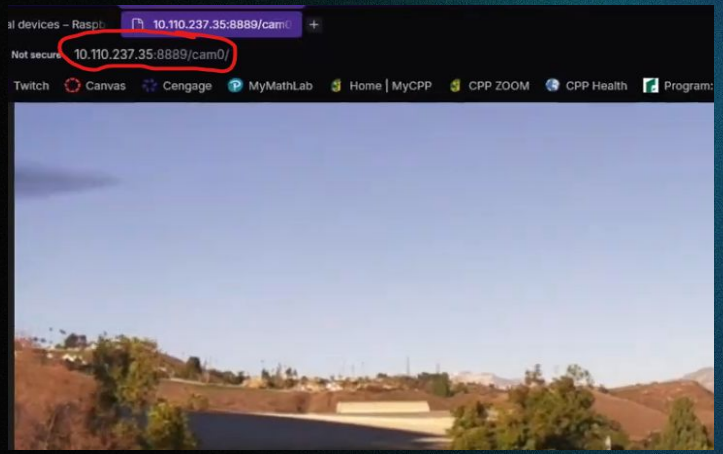
MediaMTX:

- Powerful open-source media server and proxy.
- Acts as a “media router” to handle live video /audio streams.
- Utilizes various protocols such as SRT, WebRTC, RTSP, RTMP, and HLS
- Allows for publishing, reading, proxying, recording, and playback



Viewing Live Footage:

- ipaddress:port/cam0/
- ex) 192.168.1.26:8889/cam0/
- Achieves <300 ms latency
  Reaches 30 FPS




```
crbell@crispy5:~ $ ifconfig
eth0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 2c:cf:67:b9:ef:ae  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 106

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 24  bytes 3270 (3.1 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 24  bytes 3270 (3.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

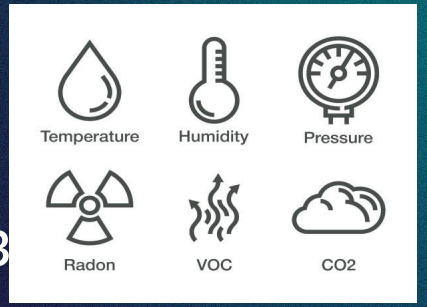
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.1.26  netmask 255.255.255.0  broadcast 192.168.1.255
        inet6 fe80::6817:4c1f:8f42:87ee  prefixlen 64  scopeid 0x20<link>
        ether 2c:cf:67:b9:ef:af  txqueuelen 1000  (Ethernet)
        RX packets 2603  bytes 348130 (339.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 4557  bytes 4879517 (4.6 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

```
crbell@crispy5:~/mediamtx $ ./mediamtx
2025/12/14 19:23:04 INF MediaMTX v1.15.3
2025/12/14 19:23:04 INF configuration loaded from /home/crbell/mediamtx/mediamtx.yml
2025/12/14 19:23:04 INF [RTSP] listener opened on :8554 (TCP), :8000 (UDP/RTP), :8001 (UDP/RTCP)
2025/12/14 19:23:04 INF [RTMP] listener opened on :1935
2025/12/14 19:23:04 INF [HLS] listener opened on :8888
2025/12/14 19:23:04 INF [WebRTC] listener opened on :8889 (HTTP)  :8189 (ICE/UDP)
2025/12/14 19:23:04 INF [SRT] listener opened on :8890 (UDP)
```

Sense Module:

Environmental Sensors:

- Air Quality Sensors - PM2.5, PM10, CO2, VOCs
- Gas Sensors - CO, NOx, SO2, H2S, CH4, C3H8, O3 NH3, etc.
- Weather Sensors - Temperature, pressure, humidity





Goal:

- Monitor for potentially dangerous and/or fluctuating environmental factors.



Software (Codes): BME280 :

```
import time
import board
from adafruit_bme280 import basic as adafruit_bme280

# --- Configuration ---
# Sea level pressure standard (matches your Arduino code)
SEALEVELPRESSURE_HPA = 1013.25

def setup():
    print("BME280 test")

    # Create library object using our Bus I2C port
    i2c = board.I2C()  # uses board.SCL and board.SDA

    try:
        # Address is usually 0x77. If that fails, try 0x76.
        bme = adafruit_bme280.Adafruit_BME280_I2C(i2c, address=0x77)
    except ValueError:
        try:
            # Fallback to 0x76 if 0x77 fails
            bme = adafruit_bme280.Adafruit_BME280_I2C(i2c, address=0x76)
        except ValueError:
            print("Could not find a valid BME280 sensor, check wiring!")
            return None

    # Set the sea level pressure for altitude calculation
    bme.sea_level_pressure = SEALEVELPRESSURE_HPA

    print("-- Default Test --")
    return bme
```

```
def print_values(bme):
    # Temperature (Arduino uses readTemperature())
    print(f"Temperature = {bme.temperature:.2f} *C")

    # Pressure
    # Note: Arduino library returns Pa, so it divides by 100.
    # Python library returns hPa directly, so NO division is needed.
    print(f"Pressure = {bme.pressure:.2f} hPa")

    # Altitude
    # Calculated based on the sea_level_pressure we set earlier
    print(f"Approx. Altitude = {bme.altitude:.2f} m")

    # Humidity
    print(f"Humidity = {bme.relative_humidity:.2f} %")

    print() # Empty line

def loop(bme):
    # Standard Arduino-like loop
    while True:
        print_values(bme)
        time.sleep(1) # Delay 1000ms

# --- Main Execution ---
if __name__ == "__main__":
    sensor = setup()

    if sensor is not None:
        try:
            loop(sensor)
        except KeyboardInterrupt:
            print("\nExiting...")
```

Software (Codes): BME280 :

```
Temperature = 24.76 *C
Pressure = 1004.53 hPa
Approx. Altitude = 72.83 m
Humidity = 62.06 %

Temperature = 24.76 *C
Pressure = 1004.55 hPa
Approx. Altitude = 72.66 m
Humidity = 62.01 %

Temperature = 24.75 *C
Pressure = 1004.54 hPa
Approx. Altitude = 72.77 m
Humidity = 62.02 %
```

Software (Codes):

MiCS-5524 :

```
import time
import math
from gpiozero import DigitalOutputDevice, MCP3008

# --- Constants & Configuration ---
CALIBRATION_TIME = 2  # Minutes
ADC_CHANNEL = 0       # Equivalent to A0 (Channel 0 on MCP3008)
POWER_PIN = 2         # GPIO 2 (Physical pin 3)

# Gas Type Constants (IDs)
CO = 0
CH4 = 1
C2H5OH = 2
H2 = 3
NH3 = 4
NO2 = 5

class DFRobot_MICS_ADC:
    def __init__(self, adc_channel, power_pin):
        # Initialize ADC (assumes MCP3008 is connected via SPI)
        self.adc = MCP3008(channel=adc_channel)
        # Initialize Power Pin
        self.power = DigitalOutputDevice(power_pin)
        self.is_sleeping = True
        self.__start_time = 0

        # NOTE: These are approximate default sensitivity values.
        # In the C++ library, these are hardcoded lookup tables.
        # We use a base resistance (R0) approximation here.
        self.r0 = -1

    def begin(self):
        # On Pi, if the object initialized without error, we assume it's connected
        return True

    def get_power_state(self):
        return 0 if self.is_sleeping else 1
```

```
    def wake_up_mode(self):
        self.power.on()
        self.is_sleeping = False
        # Reset internal timer for warm up reference
        self.__start_time = time.time()

    def sleep_mode(self):
        self.power.off()
        self.is_sleeping = True

    def warm_up_time(self, minutes):
        # Returns False if still warming up, True if done
        elapsed = (time.time() - self.__start_time)
        required_seconds = minutes * 60
        return elapsed >= required_seconds

    def get_gas_data(self, gas_type):
        """
        Reads voltage, calculates resistance (Rs), and estimates PPM.
        Note: Real accuracy requires calibrating R0 in clean air first.
        """
        # Read voltage (0.0 to 1.0 represents 0V to 3.3V)
        voltage = self.adc.value * 3.3

        # Avoid division by zero
        if voltage <= 0: return 0.0

        # Calculate Sensor Resistance (Rs) assuming a 10k divider resistor
        # Standard circuit: Vout = Vcc * (R_load / (Rs + R_load))
        # Solved for Rs: Rs = (Vcc * R_load / Vout) - R_load
        r_load = 10000  # 10k Ohm
        rs = (3.3 * r_load / voltage) - r_load
        if rs < 0: rs = 0

        # Placeholder ratios (Rs/R0) -> PPM conversion
        # The C++ library uses specific exponential curves for each gas.
```

Software (Codes):

MiCS-5524 :

```
        # Simplified Approximation: PPM = A * (Rs/R0)^B
        # Without calibration, we return a raw scaled value to show it works.

        return rs / 100.0 # Returning Raw Resistance scaled down for display

# --- Main Program Setup ---

# Initialize the sensor object
mics = DFRobot_MICS_ADC(ADC_CHANNEL, POWER_PIN)

def setup():
    print("Initializing...")

    # Attempt to initialize
    while not mics.begin():
        print("No devices found! Please check the connections.")
        time.sleep(1)

    print("Device connected successfully!")

    # Check power state
    mode = mics.get_power_state()
    if mode == 0: # 0 is sleep/off
        mics.wake_up_mode()
        print("Sensor woken up successfully!")
    else:
        print("Sensor is already in wake-up mode.")

    # Wait for warm-up
    # Note: warm_up_time returns True when FINISHED, False when busy
    print(f"Warming up for {CALIBRATION_TIME} minutes...")
    while not mics.warm_up_time(CALIBRATION_TIME):
        # We print a dot or message to keep the user updated without spamming
        print("Please wait until the warm-up time is over!", end='\r')
        time.sleep(1)
    print("\nSensor is ready for use.")

def loop():
    while True:
        # Read gas data
        co_concentration = mics.get_gas_data(CO)
        ch4_concentration = mics.get_gas_data(CH4)
        c2h5oh_concentration = mics.get_gas_data(C2H5OH)
        h2_concentration = mics.get_gas_data(H2)
        nh3_concentration = mics.get_gas_data(NH3)
        no2_concentration = mics.get_gas_data(NO2)

        # Print data
        print(f"CO (Carbon Monoxide): {co_concentration:.1f} PPM")
        print(f"CH4 (Methane): {ch4_concentration:.1f} PPM")
        print(f"C2H5OH (Ethanol): {c2h5oh_concentration:.1f} PPM")
        print(f"H2 (Hydrogen): {h2_concentration:.1f} PPM")
        print(f"NH3 (Ammonia): {nh3_concentration:.1f} PPM")
        print(f"NO2 (Nitrogen Dioxide): {no2_concentration:.1f} PPM")
        print()

        time.sleep(1)

        # mics.sleep_mode() # Uncomment to sleep after reading

# --- Execution ---
if __name__ == "__main__":
    try:
        setup()
        loop()
    except KeyboardInterrupt:
        print("\nExiting...")
        mics.sleep_mode()
```

Software (Codes):

MiCS-5524 :

```
CH4 (Methane): 19890.8 PPM
C2H5OH (Ethanol): 0.0 PPM
H2 (Hydrogen): 0.0 PPM
NH3 (Ammonia): 315.0 PPM
NO2 (Nitrogen Dioxide): 0.0 PPM

CO (Carbon Monoxide): 472.9 PPM
CH4 (Methane): 24112.8 PPM
C2H5OH (Ethanol): 129.1 PPM
H2 (Hydrogen): 183.1 PPM
NH3 (Ammonia): 377.7 PPM
NO2 (Nitrogen Dioxide): 0.0 PPM
```

Software (Codes):

MQ-2 :

```
import time
import math
from gpiozero import MCP3008
from RPLCD.gpio import CharLCD
from RPi import GPIO

# --- Hardware Configuration ---
# ADC Configuration (MCP3008)
# Connect Sensor Analog Pin -> MCP3008 Channel 0
adc = MCP3008(channel=0)

# LCD Configuration (Assumes 16x2 Display)
# You must wire these GPIO pins to your LCD
lcd = CharLCD(cols=16, rows=2, pin_rs=26, pin_e=19, pins_data=[13, 6, 5, 11], numbering_mode=GPIO.BCM)

# --- Sensor Constants ---
RL_VALUE = 5.0             # Load resistance in kilo ohms
RO_CLEAN_AIR_FACTOR = 9.83 # RO_CLEAR_AIR_FACTOR=(Sensor resistance in clean air)/RO

# --- Software Constants ---
CALIBARAION_SAMPLE_TIMES = 50
CALIBRATION_SAMPLE_INTERVAL = 0.5  # Seconds (500ms in Arduino)
READ_SAMPLE_INTERVAL = 0.05        # Seconds (50ms in Arduino)
READ_SAMPLE_TIMES = 5

# --- Gas Constants ---
GAS_LPG = 0
GAS_CO = 1
GAS_SMOKE = 2

# --- Curves ---
# Format: {x, y, slope}
# These match your Arduino definitions exactly
LPGCurve = [2.3, 0.21, -0.47]
COCurve = [2.3, 0.72, -0.34]
SmokeCurve = [2.3, 0.53, -0.44]
```

```
# Global Ro variable (init to 10k)
Ro = 10.0

# --- Helper Functions ---

def MQResistanceCalculation(raw_adc):
    """
    Input: raw_adc (0.0 to 1.0 from gpiozero, which we scale to 0-1023 equivalent)
    Output: Sensor resistance
    """
    # gpiozero returns 0.0 to 1.0. We map this to 0-1023 to match Arduino math.
    raw_val = raw_adc * 1023.0

    # Prevent division by zero if sensor is disconnected (reading 0)
    if raw_val <= 0:
        return 0.0
    if raw_val >= 1023:
        return 0.0

    return float(RL_VALUE * (1023.0 - raw_val) / raw_val)

def MQCalibration():
    val = 0.0
    for i in range(CALIBARAION_SAMPLE_TIMES):
        # Read from ADC
        val += MQResistanceCalculation(adc.value)
        time.sleep(CALIBRATION_SAMPLE_INTERVAL)

    val = val / CALIBARAION_SAMPLE_TIMES
    val = val / RO_CLEAN_AIR_FACTOR
    return val

def MQRead():
    rs = 0.0
    for i in range(READ_SAMPLE_TIMES):
        rs += MQResistanceCalculation(adc.value)
        time.sleep(READ_SAMPLE_INTERVAL)
```

Software (Codes):

MQ-2 :

```
    rs = rs / READ_SAMPLE_TIMES
    return rs

def MQGetPercentage(rs_ro_ratio, pcurve):
    """
    Uses the curve data to calculate PPM.
    Formula: 10 ^ ( ((log(rs_ro_ratio) - pcurve[1]) / pcurve[2]) + pcurve[0] )
    """
    # Check for math domain errors (log of 0 or negative)
    if rs_ro_ratio <= 0:
        return 0.0

    log_value = math.log(rs_ro_ratio)
    exponent = ((log_value - pcurve[1]) / pcurve[2]) + pcurve[0]
    return math.pow(10, exponent)

def MQGetGasPercentage(rs_ro_ratio, gas_id):
    if gas_id == GAS_LPG:
        return MQGetPercentage(rs_ro_ratio, LPGCurve)
    elif gas_id == GAS_CO:
        return MQGetPercentage(rs_ro_ratio, COCurve)
    elif gas_id == GAS_SMOKE:
        return MQGetPercentage(rs_ro_ratio, SmokeCurve)
    return 0

# --- Main Logic ---

def setup():
    global Ro
    print("Calibrating...")
    lcd.clear()
    lcd.write_string("Calibrating...")

    Ro = MQCalibration()
    print("Calibration is done...")
    print(f"Ro={Ro:.2f} kohm")

    lcd.clear()
    lcd.write_string("Done! Ro=")
    lcd.cursor_pos = (1, 0)
    lcd.write_string(f"{Ro:.2f} kohm")
    time.sleep(2)

def loop():
    while True:
        # Read current resistance
        current_rs = MQRead()
        ratio = current_rs / Ro

        # Calculate PPMs
        lpg_ppm = MQGetGasPercentage(ratio, GAS_LPG)
        co_ppm = MQGetGasPercentage(ratio, GAS_CO)
        smoke_ppm = MQGetGasPercentage(ratio, GAS_SMOKE)

        # Print to Console (Serial)
        print(f"LPG: {lpg_ppm:.0f} ppm | CO: {co_ppm:.0f} ppm | SMOKE: {smoke_ppm:.0f} ppm")

        # Update LCD
        # Row 0: LPG and CO
        lcd.clear()
        lcd.cursor_pos = (0, 0)
        lcd.write_string(f"LPG:{lpg_ppm:.0f}")

        lcd.cursor_pos = (0, 9)
        lcd.write_string(f"CO:{co_ppm:.0f}")

        # Row 1: Smoke
        lcd.cursor_pos = (1, 0)
        lcd.write_string(f"SMOKE:{smoke_ppm:.0f}")

        time.sleep(0.2)

# --- Execution ---
```

Software (Codes):

MQ-2 :

```
# --- Execution ---
if __name__ == "__main__":
    try:
        setup()
        loop()
    except KeyboardInterrupt:
        print("\nExiting...")
        lcd.clear()
        lcd.write_string("Stopped")
```

```
LPG:-5275ppm    CO:0ppm    SMOKE:-28348ppm
LPG:0ppm    CO:8712ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:2176ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:-16564ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:-11344ppm    SMOKE:0ppm
LPG:-10564ppm    CO:-27264ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:0ppm    CO:0ppm    SMOKE:0ppm
LPG:-4083ppm    CO:26272ppm    SMOKE:
```

System Integration:

Sensor Fusion:

- Synchronize all data streams.
- Integrate visual, environmental, and terrain data to enhance situational awareness, reduce sensor error, & create a more accurate view of the surroundings.

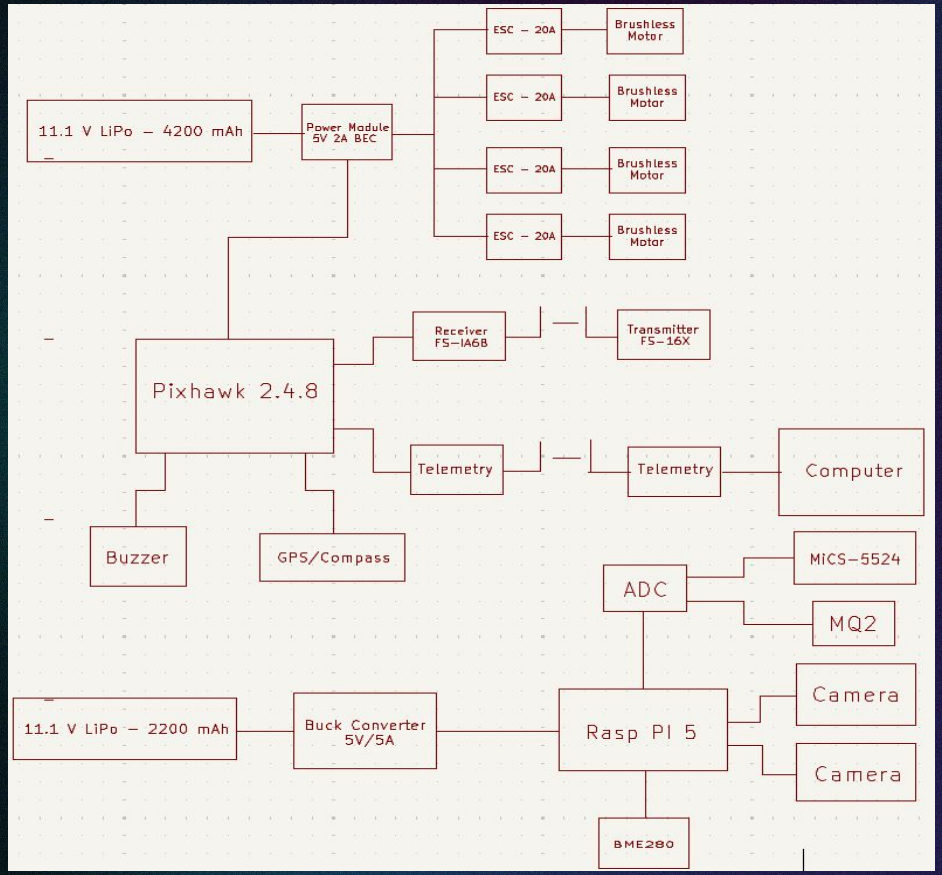
Data Logging:

- Time-stamped, geotagged sensor data which will be uploaded to an online server/database.

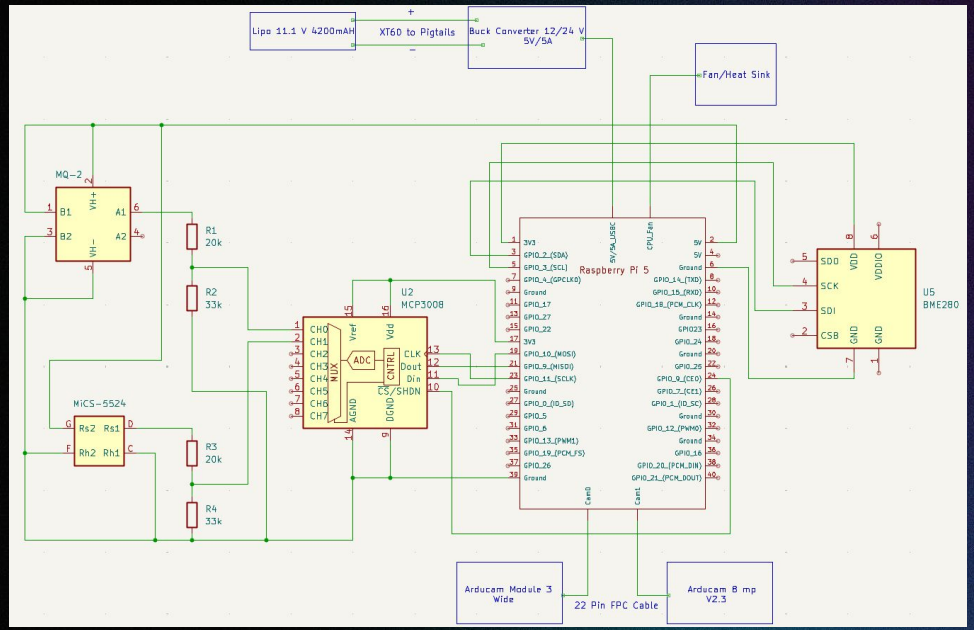
Flight Autonomy & Control Software:

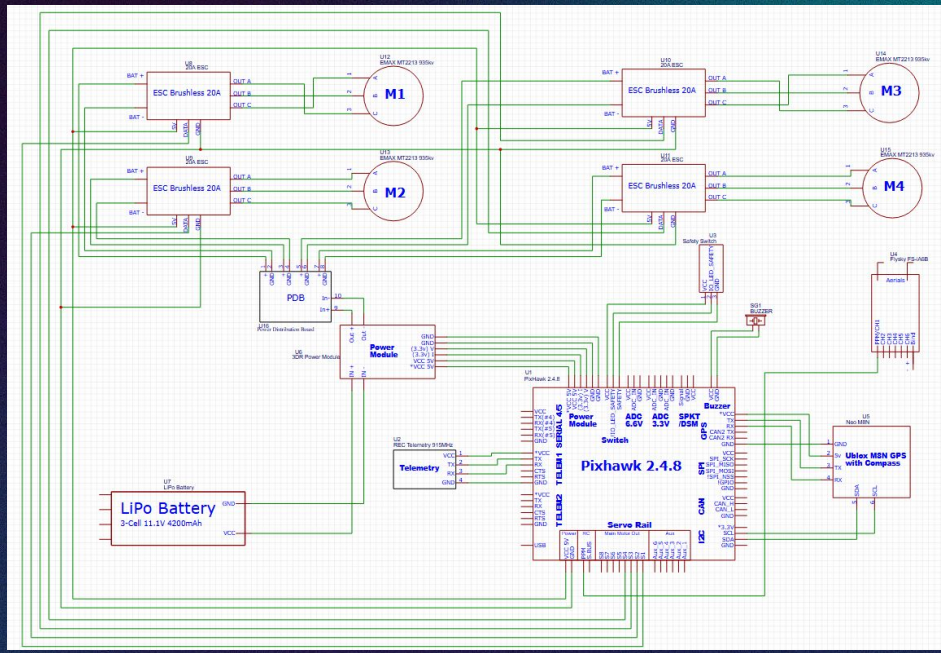
- Manual + semi-autonomous flight modes

Block Diagram:



Schematics:


Lipo 11.1 V 4200mAH
XT60 to Pigtails
Buck Converter 12/24 V 5V/5A
Fan/Heat Sink
MQ-2
R1 20k
R2 33k
R3 20k
R4 33k
MiCS-5524
U2 MCP3008
Raspberry Pi 5
U5 BME280
Arducam Module 3 Wide
22 Pin FPC Cable
Arducam 8 mp V2.3


ESC Brushless 20A
M1
M2
M3
M4
PDB
Power Module
LiPo Battery
3-Cell 11.1V 4200mAh
Pixhawk 2.4.8
Telemetry
Buzzer
Serve Rail
Ublox M8N GPS with Compass

Final Product:









Final Results:

- Successful in achieving stable flight:
- Flight time between 6 and 8 minutes depending on factors such as hovering and time under full throttle.
- The Pi is able to run anywhere between 30 and 45 minutes depending on factors such as time under full load and idling.

- Hardware specifications resulted in good robust UAV design:
- Relatively sturdy.
- Good response.
- High-power propulsion.

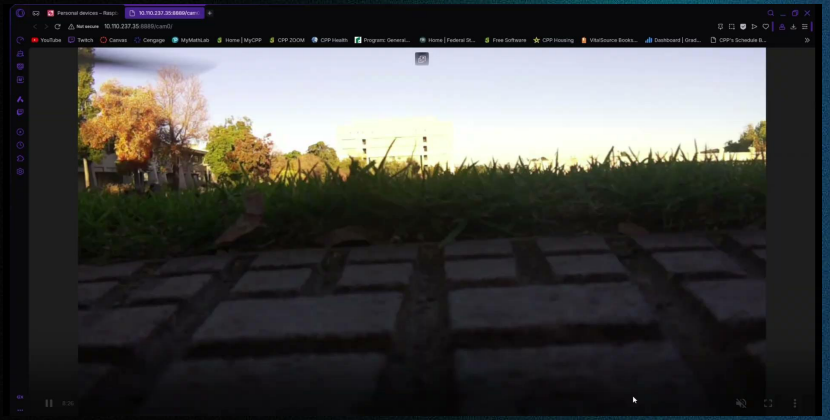
- Over budget:
- Expected cost: $450
- Actual cost: $900

Challenges & Considerations:

- Power management with multiple sensors
- Weight vs. flight time trade-offs
- Sensor calibration and accuracy
- Data bandwidth and storage
- Weatherproofing

Demos:



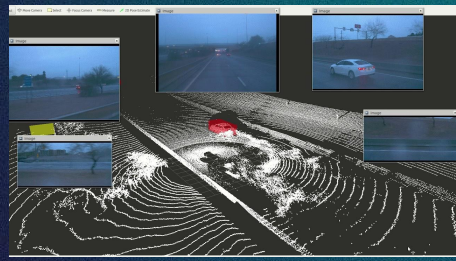






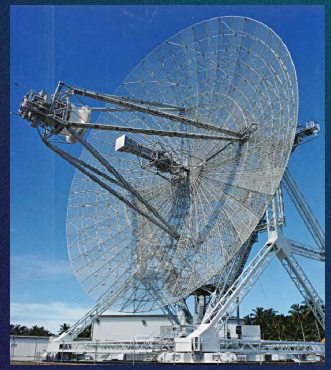
Future Possib. Radar vs LIDAR:





LIDAR:

- High-resolution mapping
- Affected by fog, rain, dust
- Expensive - used by Waymo (lasers detect rain)

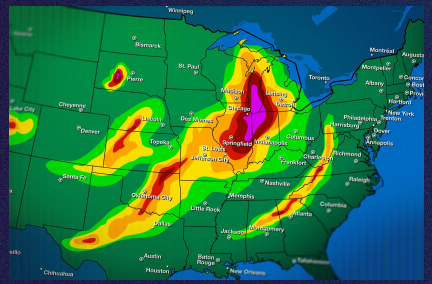


RADAR:

- Lower resolution but better in poor weather
- More robust and cost-effective

Why we would choose RADAR:

- Versatility in different environments
- Real-world viability and affordability



Future Work & Improvements:

- Thermal Imaging Camera - MLX90640
- 10.525GHz Doppler RADAR
- Geiger Counter
- Payload Hook or Delivery Mechanism
- Gimbal Mount
- LEDs/Lasers

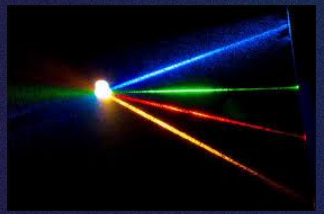






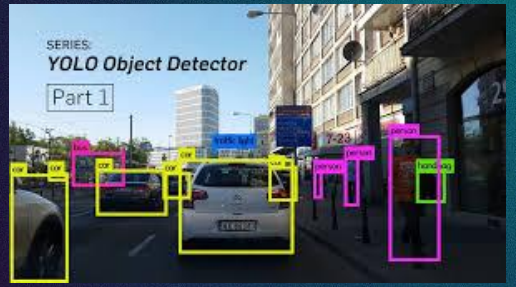






Future Work & Improvements:

- AI-based object recognition & avoidance (rescue targets, terrain classification)
- Improved battery life & solar-assisted flight
- Deployable swarm of drones for large-area coverage
- Cloud-based real-time dashboards









Conclusion:

END

Video Presentation